PROJECT REPORT: AI-POWERED GESTURE CONTROL

1. Abstract

This project details the development of a real-time gesture recognition system designed to interface human hand movements with microcontroller-based hardware. By utilizing computer vision, specifically the MediaPipe landmark detection model (implemented via the CvZone library), the system achieves accurate, high-fidelity hand tracking. We implement a custom Euclidean distance-based similarity algorithm to allow users to "train" the system on specific postures, which then act as triggers for external LED indicators. This report outlines the engineering process, the mathematical logic of the pose-matcher, and the integration between Python and Arduino via Serial communication.

2. Introduction

Human-Computer Interaction (HCI) is shifting toward touchless, intuitive interfaces. Traditional hardware controls often require physical contact, which can be limiting in industrial, medical, or specialized environments. This system addresses this by leveraging a webcam to extract high-dimensional coordinate data from the human hand, which is then processed through a machine learning-based geometry matcher to execute physical actions. The primary goal is to provide a low-latency, responsive, and re-configurable interface for hardware control.

3. Theoretical Framework

3.1 Computer Vision & Hand Tracking

The system relies on the MediaPipe Hands model, a robust tracking pipeline that identifies 21 3D landmarks on the human hand. This model utilizes a palm detector and a landmark model to ensure consistent tracking regardless of hand orientation, scale, or background clutter.

3.2 Euclidean Geometry for Gesture Matching

Gesture recognition is treated here as a classification problem. We represent a gesture as a feature vector P in 10-dimensional space (the x and y coordinates of the 5 fingertips). To compare a real-

time pose (P_c) with a stored "Target" pose (P_t), we calculate the Euclidean distance: $dist = \sqrt{\Sigma(P_{c,i} - P_{t,i})^2}$. A distance threshold ($\tau = 150$) is applied to determine if the pose is a "match."

4. Implementation Code

```
# --- IMPORTS ---
import cv2                 # OpenCV: Library for video processing
import serial              # PySerial: Enables PC-to-Arduino communication
import time                # Time: Manages serial port stabilization
import numpy as np         # NumPy: Tools for vector distance calculations
from cvzone.HandTrackingModule import HandDetector # Hand Tracking: Uses MediaPipe
engine

# --- INITIALIZATION ---
arduino = serial.Serial('COM5', 9600, timeout=1) # Connect to Arduino on COM5
time.sleep(2)              # Wait for serial connection to stabilize
cap = cv2.VideoCapture(0)  # Open default camera
detector = HandDetector(maxHands=1, detectionCon=0.8) # Detection confidence 0.8
target_pose = None         # Storage for the saved gesture vector

# --- FUNCTIONS ---
def get_hand_geometry(lmList):
    # Extract coordinates for the 5 fingertips
    fingertips = [lmList[4], lmList[8], lmList[12], lmList[16], lmList[20]]
    # Flatten list into a 1D array for vector comparison
    return np.array(fingertips).flatten()

# --- MAIN LOOP ---
while True:
    success, frame = cap.read() # Capture frame
    if not success: break
    frame = cv2.flip(frame, 1) # Mirror reflection
    hands, frame = detector.findHands(frame, draw=False) # Perform detection
    key = cv2.waitKey(1) & 0xFF # Listen for keyboard commands

    if hands:
        lmList = hands[0]['lmList'] # Get coordinates of 21 landmarks
        current_pose = get_hand_geometry(lmList) # Convert to math vector

        if key == ord('s'): target_pose = current_pose # Save gesture
        elif key == ord('r'): target_pose = None # Reset gesture

        if target_pose is not None:
            dist = np.linalg.norm(current_pose - target_pose) # Euclidean Distance
            # If distance < 150, send 'G' (Match), else 'R' (No Match)
            arduino.write(b'G' if dist < 150 else b'R')
    else:
        arduino.write(b'R') # Default to 'R' if no hand detected

    cv2.imshow("Gesture Recognition GUI", frame) # Display GUI
    if key == ord('q'): break # Quit loop

cap.release() # Release resources
```

```
cv2.destroyAllWindows() # Clear memory
```

5. Conclusion

This project provides a reliable, low-latency solution for gesture-based hardware control. By combining the strengths of MediaPipe for landmark identification and NumPy for vector analysis, we created an interface that is both accessible and mathematically rigorous.